



RAJIV GANDHI UNIVERSITY OF KNOWLEDGE TECHNOLOGIES

(A.P. Government Act 18 of 2008)

RGUKT – Nuzvid, Andhra Pradesh – 521202

LearnTube

AI-Powered Dynamic Course Generator from YouTube Videos

Mini Project Report for Bachelor of Technology in
Computer Science and Engineering

Submitted by:

N200388 – N. Lakshmana

N210041 – K. Yerni Sagar

N210597 – P. Karthik

N211016 – M. Sudhakar Naik

Under the Esteem Guidance of

Mrs. M. Jyothi

Asst. Professor, Department of CSE

Department of Computer Science and Engineering

Rajiv Gandhi University of Knowledge Technologies, Nuzvid

Academic Year: 2025 – 2026



**RAJIV GANDHI UNIVERSITY OF KNOWLEDGE
TECHNOLOGIES**

(A.P. Government Act 18 of 2008)

RGUKT-NUZVID, Krishna Dist - 521202

CERTIFICATE OF COMPLETION

This is to certify that the work entitled, **“LEARNTUBE : AI-POWERED DYNAMIC COURSE GENERATOR FROM YOUTUBE VIDEOS”** is the Bonafide work of **N. Lakshmana** with ID No: **N200388**, **K. Yerni Sagar** with ID No: **N210041**, **P. Karthik** with ID No: **N210597** and **M. Sudhakar Naik** with ID No: **N211016** carried out under my guidance and supervision for the Minor Project of Bachelor of Technology in the department of Computer Science and Engineering under RGUKT IIIT Nuzvid. This work is done during the academic session **December 2025 – April 2026**, under our guidance.

Mrs. M. Jyothi

Assistant Professor,
Department of CSE,
RGUKT – Nuzvid

Mr. A. Uday Kumar

Head of Department,
Department of CSE,
RGUKT – Nuzvid



**RAJIV GANDHI UNIVERSITY OF KNOWLEDGE
TECHNOLOGIES**

(A.P. Government Act 18 of 2008)

RGUKT-NUZVID, Krishna Dist – 521202

CERTIFICATE OF EXAMINATION

This is to certify that the work entitled, “**LEARNTUBE: AI-POWERED DYNAMIC COURSE GENERATOR FROM YOUTUBE VIDEOS**” is the Bonafide work of **N. Lakshmana** with ID No: **N200388**, **K. Yerni Sagar** with ID No: **N210041**, **P. Karthik** with ID No: **N210597** and **M. Sudhakar Naik** with ID No: **N211016** and hereby accord our approval of it as a study carried out and presented in a manner required for its acceptance in Minor Project of Bachelor of Technology for which it has been submitted. This approval does not necessarily endorse or accept every statement made, opinion expressed or conclusion drawn, as recorded in this thesis. It only signifies the acceptance of this thesis for the purpose for which it has been submitted.

Mrs. M. Jyothi

Assistant Professor,
Department of CSE,
RGUKT – Nuzvid

Project Examiner

1. Mr. K. Lakshmi Kanth
2. Mrs. B. Padma Bai
Assistant Professor,
Department of CSE,
RGUKT – Nuzvid



**RAJIV GANDHI UNIVERSITY OF KNOWLEDGE
TECHNOLOGIES**

(A.P. Government Act 18 of 2008)

RGUKT-NUZVID, Krishna Dist - 521202

DECLARATION

We **N.LAKSHMANA** with ID No: **N200388** and **K.YERNI SAGAR** with ID No: **N210041** and **P.KARTHIK** with ID No: **N210597** and **M.SUDHAKAR NAIK** with ID No: **N211016** hereby declare that the project report entitled “**LEARNTUBE: AI-POWERED DYNAMIC COURSE GENERATOR FROM YOUTUBE VIDEOS**” done by us under the guidance of **Mrs. M. Jyothi** is submitted for the partial fulfillment of the Minor Project for the Bachelor of Technology in Computer Science and Engineering during the academic session **December 2025 – April 2026** at RGUKT-Nuzvid.

We also declare that this project is a result of our own effort and has not been copied or imitated from any source. Citations from any websites are mentioned in the references. The results embodied in this project report have not been submitted to any other university or institute for the award of any degree or diploma.

Date : _____

Place : _____



**RAJIV GANDHI UNIVERSITY OF KNOWLEDGE
TECHNOLOGIES**

(A.P. Government Act 18 of 2008)

RGUKT-NUZVID, Krishna Dist – 521202

ACKNOWLEDGEMENT

We would like to express our profound gratitude and deep regards to our guide **Mrs. M. Jyothi**, Assistant Professor (C), Department of Computer Science and Engineering, RGUKT-Nuzvid, for her exemplary guidance, monitoring, and constant encouragement throughout the B.Tech course. We shall always cherish the time spent during the course of this work due to the invaluable guidance received throughout the project.

We are extremely grateful for the confidence bestowed upon us and for entrusting us with our project entitled “**LearnTube: AI-Powered Dynamic Course Generator from YouTube Videos**”.

We express our gratitude to **Mr. A. Uday Kumar** (HOD of CSE) and other faculty members for being a source of inspiration and constant encouragement, which helped us in completing the project successfully.

We also thank the **open-source community** behind the Groq LLM platform, the sentence-transformers library, the youtube-transcript-api and yt-dlp projects, the FastAPI and Next.js teams, and the MongoDB Atlas team — this project would not be possible without their contributions.

Finally, we thank our family and friends for their constant support and motivation.

ABSTRACT

Students who rely on YouTube for free education face a fundamental problem: video content is unstructured, undocumented, and unverifiable. **LearnTube** solves this by accepting any educational YouTube URL and automatically converting it into a complete structured course. The system extracts the video transcript through a four-strategy waterfall - *youtube-transcript-api*, Google timedtext API, *yt-dlp*, and Whisper ASR as a final fallback and supports multilingual videos through automatic Groq-powered translation. Semantic chapter boundaries are detected by embedding transcript blocks using *all-MiniLM-L6-v2* and identifying topic shifts via cosine similarity thresholding. Structured course content - lessons, summaries, key concepts, and quizzes - is then generated using the Groq Llama-3.3-70b language model. The platform handles long-form educational videos through an asynchronous job pipeline, caches all generated courses in MongoDB for instant reuse, and completes the learning lifecycle with a graded exam and a PDF certificate issued on passing. Built on a Python FastAPI modular monolith backend and a Next.js frontend with NextAuth authentication, LearnTube transforms any free YouTube video into a certifiable, professional learning experience with zero manual effort from the learner.

CONTENTS

1	Introduction	1
1.1	Motivation for the Work	1
1.2	Real-World Applications	2
2	Literature Review	3
2.1	Research Gaps Identified	4
3	System Architecture and Methodology	5
3.1	Proposed Solution	5
3.2	System Architecture	5
3.3	Experimental Setup	7
3.3.1	Dataset	7
3.3.2	Hardware Requirements	7
3.3.3	Software Requirements	8
3.4	Methodology	8
3.4.1	Course Generation Flowchart	9
3.4.2	Explanation of Course Generation Flow	9
3.4.3	Explanation of Exam Submission and Certificate Generation Flow .	12
3.4.4	Data Acquisition and Feature Extraction	14
3.4.5	Preprocessing and Normalization	14
3.4.6	Model Training and Inference	14
3.4.7	Algorithm	16
4	Results and Discussion	18
4.1	Training and Validation Progress	18
4.2	Performance Evaluation Metrics	18
4.3	Confusion Matrix Analysis	19
4.4	Comparison with Existing Approaches	20
5	Conclusion and Future Work	21
5.1	Conclusion	21
5.2	Limitations	22
5.3	Future Work	22

LIST OF FIGURES

Figure-1 : LearnTube four-layer system architecture	5
Figure-2 : Course generation pipeline flowchart	9
Figure-3 : Exam submission and certificate generation flowchart	12

LIST OF TABLES

Table-1 : Existing Systems and Their Limitations	3
Table-2 : Software modules and functional purpose	8
Table-3 : Functional test results	18
Table-4 : Feature comparison with existing systems	20

CHAPTER 1

INTRODUCTION

LearnTube is an AI-powered system designed to transform unstructured educational YouTube videos into fully structured and certifiable learning experiences. It addresses key challenges such as lack of organization, absence of assessments, and language barriers in free on-line learning content. By automatically extracting transcripts, detecting semantic topics, and generating lessons, summaries, and quizzes, the platform creates a complete course from a single video URL. The system leverages advanced language models and embedding techniques to ensure meaningful content structuring and multilingual support. Ultimately, LearnTube enables accessible, self-paced, and verifiable education for learners without requiring any manual effort.

1.1 Motivation for the Work

Digital learning has undergone a fundamental transformation in recent years. YouTube hosts over 800 hours of new video content every minute, a significant portion of which is educational—covering programming, mathematics, science, language learning, and nearly every academic discipline. Millions of students, particularly in developing economies, rely heavily on YouTube for self-directed learning due to the high cost of paid educational platforms.

Key Challenges in Current Learning Systems

However, this reliance on free video content introduces several critical challenges that affect learning effectiveness:

- **Absence of Structure:** Educational content is presented as a continuous stream without syllabus design, chapter segmentation, organized notes, or defined learning objectives.
- **Lack of Assessment:** There is no built-in mechanism to evaluate understanding, track progress, or validate knowledge acquisition.
- **Language Barriers:** High-quality content is often available only in specific languages, limiting accessibility for non-native speakers.

Core Problem Statement

How can the vast repository of free educational video content on YouTube be automatically transformed into structured, assessable, and certifiable learning experiences—without requiring any manual effort from the learner?

Motivation Behind the Project

This project is driven by the following key observations:

1. No existing open-source system provides a complete pipeline from YouTube URL to a structured course with quizzes and certification.
2. Modern Large Language Models (LLMs) are capable of generating high-quality educational content but lack integration into a seamless, user-friendly platform.
3. Semantic chapter detection using sentence embeddings is technically feasible but has not been effectively utilized in real-world course generation systems.

1.2 Real-World Applications

Practical Use Cases of LearnTube

The LearnTube system can be applied across multiple real-world scenarios:

- **Self-Paced Learning:** Enables learners to convert any educational video into a structured course for systematic study.
- **Accessible Education:** Provides underprivileged and rural students access to certification-quality learning at zero cost.
- **Corporate Training:** Converts internal training videos into trackable and certifiable modules for employee onboarding and upskilling.
- **Multilingual Learning:** Automatically translates non-English content into English, making global knowledge accessible.
- **Academic Support:** Helps educators integrate YouTube content into formal teaching without manual content creation.
- **Exam Preparation:** Assists students in generating notes and quizzes for efficient revision.
- **Lifelong Learning:** Supports professionals in continuously upgrading skills through structured learning from online content.

CHAPTER 2

LITERATURE REVIEW

Table-1 : Existing Systems and Their Limitations

System	Approach	Key Limitations
Coursera / Udemy	Structured online courses with quizzes and certificates	All content is manually authored and platform-bound; no mechanism to convert external video into a course; expensive subscription or per-course fees; not accessible to all learners
YouTube Chapters	Timestamp-based navigation within a single video	Entirely manual—requires creator to add chapters; no notes, no quiz, no certificate; no cross-video structure; no AI involvement
Summify / TL-DRthis	Automatic video or article summarisation	Produces only a flat summary paragraph; no lesson hierarchy, no quiz generation, no certification; cannot handle playlists or courses
Otter.ai / Rev	Automatic speech recognition and transcription	Stops at transcript generation; no downstream course structure, no quiz, no certificate; requires paid subscription for full access
ChatGPT (manual)	User pastes transcript and requests notes or quiz	Entirely manual workflow; no YouTube integration; no persistent course storage; no exam grading; no certificate; no multilingual pipeline
Khan Academy / Khanmigo	Structured AI-assisted learning with exercises	Works only on platform content; cannot ingest arbitrary YouTube videos; no open API for course generation from external content
Notion AI / Mem	AI-assisted note-taking from pasted text	Requires manual transcript copy-paste; no YouTube integration; no quiz generation; no exam or certificate; not a full learning system

2.1 Research Gaps Identified

Critical analysis of the above systems reveals five unaddressed challenges that LearnTube specifically targets:

1. **No end-to-end pipeline from YouTube URL to certified course.** Every existing tool addresses at most one stage of the learning pipeline (transcription, or summarisation, or quiz generation) but no system chains all stages — from URL input through transcript extraction, semantic structuring, quiz generation, exam grading, and certificate issuance — into a single automated workflow.
2. **Multilingual transcript extraction with LLM-based translation is unaddressed.** Existing tools either require English captions or use YouTube’s built-in translation, which is aggressively rate-limited for server-side requests. No open-source course generation system implements a chunk-based LLM translation fallback for non-English educational content.
3. **Semantic chapter detection from transcript text is underexplored.** YouTube’s manual chapter system requires creator input. Automatic segmentation using sentence-transformer embeddings and cosine similarity thresholding — applied specifically to educational transcript text for pedagogically meaningful chapter boundaries — has no open-source implementation.
4. **Async processing for long-form educational videos is not implemented.** Long educational lectures (1–3 hours) cause timeout failures in synchronous HTTP pipelines. No existing open-source YouTube-to-course tool implements a background job architecture with persistent status polling for long-form video processing.
5. **No unified modular system exists combining all necessary services.** Building a complete course generation system requires combining transcript extraction, content classification, chapter detection, LLM-based course generation, database caching, exam grading, and certificate generation. No single deployable open-source backend implements all of these in a clean, modular architecture.

CHAPTER 3

SYSTEM ARCHITECTURE AND METHODOLOGY

3.1 Proposed Solution

LearnTube is a modular monolith backend combined with a Next.js frontend. The system is designed around a single core pipeline: a YouTube URL enters, a complete structured course exits. Unlike standard summarisation tools that stop at transcript extraction, this project targets the full learning lifecycle — from URL input through certificate issuance — using a combination of deep learning, LLM-based generation, and asynchronous processing. By using a hybrid architecture with MongoDB caching, the solution ensures that users can access powerful course generation capabilities through a standard browser without repeatedly incurring API costs.

3.2 System Architecture

The architecture has four logical layers.

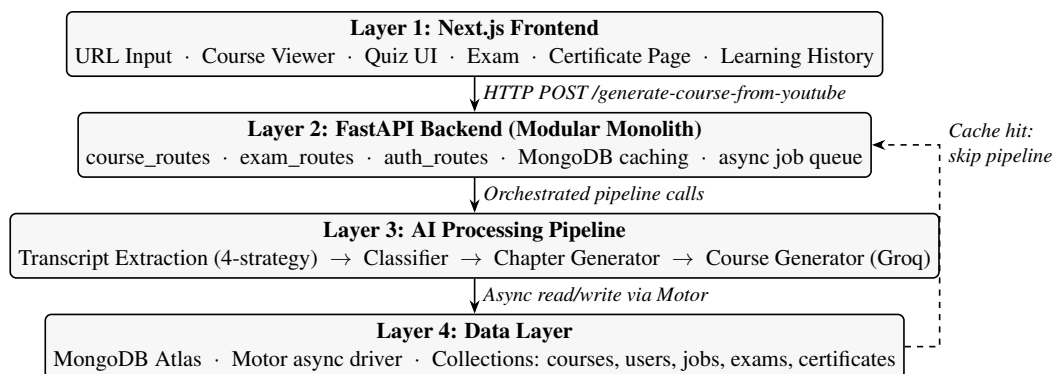


Figure-1 : LearnTube four-layer system architecture

System Architecture Explanation

The LearnTube system architecture is designed as a four-layer modular pipeline that transforms a simple YouTube URL into a complete structured course. The architecture follows a layered and modular design approach, ensuring clear separation of responsibilities across components. This design not only improves system maintainability but also allows independent scaling and upgrading of different layers without affecting the overall system.

At the top, the **Frontend Layer (Next.js)** acts as the user interface where learners interact with the system. Users can input a YouTube URL, view generated courses, navigate through lessons, attempt quizzes, take exams, and access certificates. The frontend is designed to be responsive and user-friendly, ensuring a smooth experience across devices

such as desktops and mobiles. It communicates with the backend through RESTful HTTP requests and dynamically renders content received from APIs. Additionally, it manages user sessions, handles navigation, and ensures an interactive learning environment through structured UI components.

The **Backend Layer (FastAPI modular monolith)** functions as the core controller of the system. It is responsible for handling API requests, managing authentication and authorization, coordinating course generation workflows, evaluating exams, and handling certificate generation. When a user submits a video URL, the backend orchestrates the entire workflow by first validating the input and then checking MongoDB for cached results. This caching mechanism avoids redundant processing and significantly improves system performance. If no cached data is found, the backend triggers the AI processing pipeline. The modular monolith architecture ensures that different functional modules such as courses, exams, and certificates are logically separated while still being deployed as a single service for simplicity and easier management.

The **AI Processing Pipeline Layer** serves as the intelligence core of the system. It performs transcript extraction using a multi-strategy approach to ensure robustness even when certain APIs fail. After obtaining the transcript, the system performs language detection and translation if necessary. It then classifies the content to ensure that only educational videos are processed further. The pipeline uses embedding techniques and cosine similarity to detect semantic boundaries, enabling meaningful chapter segmentation. Finally, advanced large language models generate structured course content, including lessons, summaries, key concepts, and quizzes. This layer plays a crucial role in converting unstructured video data into a well-organized and learner-friendly format.

The **Data Layer (MongoDB Atlas with Motor driver)** manages persistent storage of all system data, including users, courses, transcripts, exams, and certificates. MongoDB provides flexibility in handling semi-structured JSON data, making it suitable for storing dynamically generated course content. The Motor driver enables asynchronous database operations, which improves performance under concurrent user requests. Additionally, caching is implemented at this layer, allowing previously processed videos to bypass the AI pipeline, thereby reducing response time and computational overhead.

Core Design Invariant

The MongoDB cache is checked **before** any AI processing begins. If a course for the given `video_id` already exists, it is returned immediately — saving 20–45 seconds of processing time and all associated API costs.

3.3 Experimental Setup

3.3.1 Dataset

No external labelled dataset was used for training. LearnTube processes any public educational YouTube video dynamically at request time:

- **Video domains tested:** Python programming, DBMS, calculus, English grammar, physics lectures, competitive exam preparation
- **Video length range:** 5 minutes to 3+ hours (async endpoint handles long videos without timeout)
- **Multilingual testing:** Hindi, Spanish, and French videos — all auto-detected and translated to English via Groq Llama-3.1-8b before course generation
- **Chapter detection validation:** Visually validated across 10+ diverse educational videos of varying lengths and topic densities
- **Pre-trained model:** `all-MiniLM-L6-v2` — 22M parameter model pre-trained on 1B+ sentence pairs; used as-is with no fine-tuning

3.3.2 Hardware Requirements

Parameter	Specification
Processor	Intel Core i5 / i7 (or equivalent)
RAM	8 GB minimum
Storage	~10 GB (sentence-transformer model cache)
GPU	Not required — Whisper tiny runs on CPU with int8 quantisation
Network	Required for YouTube caption API, yt-dlp, and Groq API calls
Browser	Google Chrome / Mozilla Firefox (Next.js frontend)
Operating System	Windows / macOS / Linux

3.3.3 Software Requirements

Table-2 : Software modules and functional purpose

Layer	Technology	Purpose
Frontend	Next.js 14 (React), Tailwind CSS	UI, routing, responsive course viewer and exam interface
Authentication	NextAuth.js	Google OAuth 2.0 and email/password login with JWT sessions
Backend	Python FastAPI, Uvicorn	Modular monolith REST API server
DB Driver	Motor (async)	Non-blocking MongoDB operations via AsyncIOMotorClient
Transcript 1	<i>youtube-transcript-api</i>	Caption extraction via YouTube caption endpoint (strategy 1)
Transcript 2	<i>yt-dlp</i>	Subtitle download and VTT/JSON3 parsing (strategy 3)
ASR Fall-back	<i>faster-whisper</i> (tiny, int8)	CPU-based audio transcription for caption-less videos (strategy 4)
Embeddings	sentence-transformers (all-MiniLM-L6-v2)	384-dim sentence embeddings for chapter boundary detection
LLM (Course)	Groq SDK — Llama-3.3-70b	Structured course JSON generation: lessons, summaries, quizzes
LLM (Utils)	Groq SDK — Llama-3.1-8b-instant	Non-English translation and educational content classification
Database	MongoDB Atlas / Local	Persistent storage: courses, users, jobs, exams, certificates
Certificate	<i>reportlab</i>	PDF certificate generation with UUID-based shareable link
Node.js	$\geq 18.x$	Frontend runtime
Python	≥ 3.10	Backend runtime
MongoDB	Atlas (cloud) or local instance	Database

3.4 Methodology

The implementation follows a structured pipeline that transforms a YouTube URL into a complete structured course.

3.4.1 Course Generation Flowchart

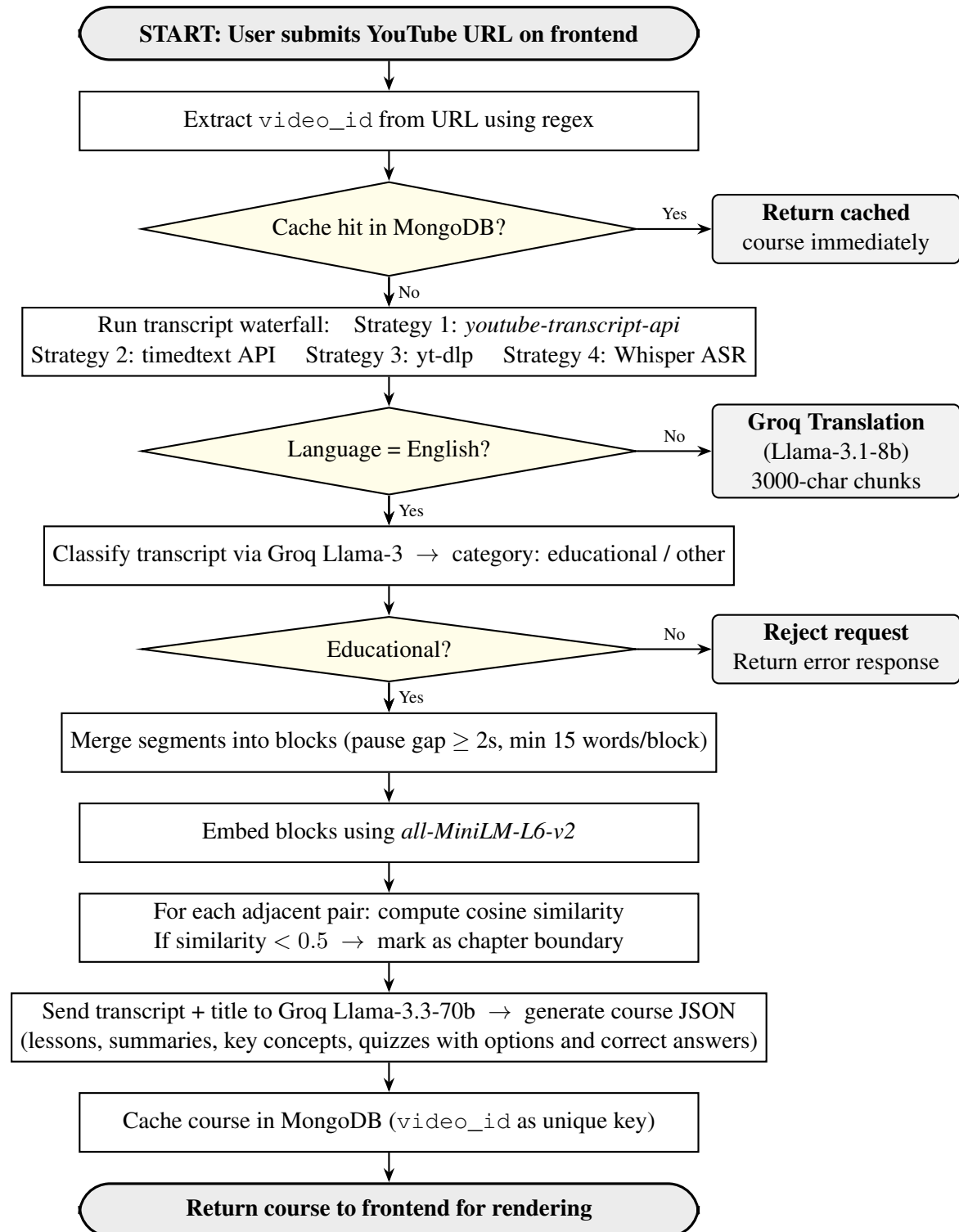


Figure-2 : Course generation pipeline flowchart

3.4.2 Explanation of Course Generation Flow

The flowchart in Figure represents the complete pipeline for generating a structured course from a YouTube video. The steps involved are as follows:

1. **User Input:** The process begins when a user submits a YouTube URL through the frontend interface.
2. **Video ID Extraction:** The system extracts the `video_id` from the URL using a regular expression.
3. **Cache Check (MongoDB):**
 - The system checks if a course for the given `video_id` already exists.
 - If found, the cached course is returned immediately (cache hit).
4. **Transcript Extraction (Waterfall Strategy):** If no cache is found, the system retrieves the transcript using multiple fallback methods:
 - *youtube-transcript-api*
 - YouTube timedtext API
 - *yt-dlp*
 - Whisper ASR (Speech-to-Text)
5. **Language Check and Translation:**
 - The system checks if the transcript is in English.
 - If not, it is translated into English using Groq Llama-3.1-8B in chunks (approx. 3000 characters).
6. **Content Classification:**
 - The transcript is classified using Groq Llama-3.
 - If the content is not educational, the request is rejected.
7. **Transcript Segmentation:**
 - The transcript is divided into blocks based on:
 - Pause gaps ≥ 2 seconds
 - Minimum 15 words per block
8. **Embedding Generation:** Each block is converted into vector embeddings using *all-MiniLM-L6-v2*.
9. **Chapter Detection:**
 - Cosine similarity is computed between adjacent blocks.
 - If similarity < 0.5 , a new chapter boundary is created.

10. Course Generation (LLM):

- The transcript and video title are sent to Groq Llama-3.3-70B.
- The model generates structured course content in JSON format, including:
 - Lessons
 - Summaries
 - Key concepts
 - Quizzes (MCQs with answers)

11. **Caching the Result:** The generated course is stored in MongoDB using `video_id` as a unique key.

12. **Final Output:** The structured course is returned to the frontend for rendering and user interaction.

Additional Discussion: The proposed pipeline is designed with scalability, reliability, and adaptability in mind. The use of a caching mechanism significantly reduces redundant processing and improves response time for repeated requests. The transcript extraction waterfall strategy ensures high availability of textual data even when certain APIs fail or are restricted. Furthermore, the integration of embedding-based semantic segmentation allows the system to understand contextual shifts within the content, resulting in meaningful chapter formation rather than arbitrary splits. The use of large language models (LLMs) enhances the overall quality of the generated course by producing human-like summaries, well-structured lessons, and assessment questions. Additionally, the modular nature of the pipeline allows easy integration of future improvements, such as better translation models, more accurate classifiers, or advanced evaluation techniques. This makes the system robust and extensible for real-world educational applications.

Conclusion: This pipeline ensures:

- Fast response using caching
- Robust transcript extraction using fallback strategies
- High-quality course generation using LLMs and semantic segmentation

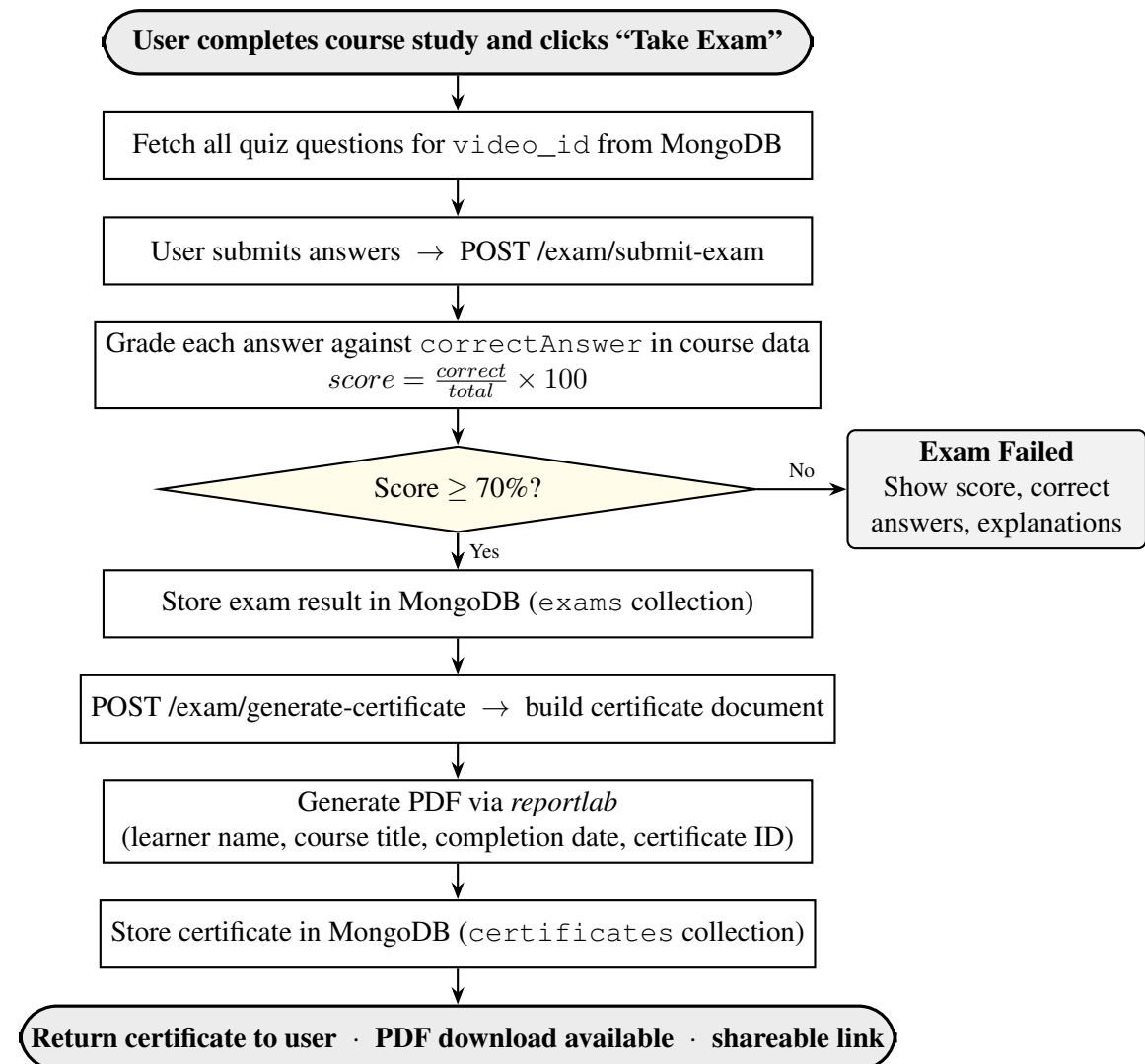


Figure-3 : Exam submission and certificate generation flowchart

3.4.3 Explanation of Exam Submission and Certificate Generation Flow

The flowchart in Figure illustrates the process of exam submission, evaluation, and certificate generation after completing a course. The steps are as follows:

1. **Exam Initiation:** After completing the course, the user clicks on the "Take Exam" option to begin the assessment.
2. **Fetch Questions:** The system retrieves all quiz questions associated with the given video_id from the MongoDB database.
3. **Answer Submission:** The user attempts the exam and submits responses through the endpoint:
POST /exam/submit-exam.
4. **Evaluation and Scoring:**

- Each submitted answer is compared with the `correctAnswer` stored in the course data.
- The final score is calculated using:

$$score = \frac{\text{correct answers}}{\text{total questions}} \times 100$$

5. Pass/Fail Decision:

- If the score is less than 70%, the exam is marked as failed.
- The system displays the score, correct answers, and explanations to the user.

6. Store Exam Result (Pass Case): If the score is greater than or equal to 70%, the exam result is stored in the MongoDB `exams` collection.

7. Certificate Generation Request: A request is sent to the endpoint:
`POST /exam/generate-certificate` to initiate certificate creation.

8. PDF Certificate Creation:

- A certificate is generated using the *reportlab* library.
- The certificate includes:
 - Learner's name
 - Course title
 - Completion date
 - Unique certificate ID

9. Store Certificate: The generated certificate is stored in the MongoDB `certificates` collection for future access.

10. Final Output:

- The certificate is returned to the user.
- A downloadable PDF is provided.
- A shareable link is also generated.

Conclusion: This workflow ensures accurate evaluation, secure result storage, and automated certificate generation, providing a complete assessment and recognition system for learners.

3.4.4 Data Acquisition and Feature Extraction

The transcript service implements a four-strategy waterfall. Each strategy is attempted in sequence; the first successful result is used and the remaining strategies are skipped.

Strategy	Method	Description
1	<i>youtube-transcript-api</i>	Fetches manual or auto-generated captions via YouTube’s caption API. Tries English first; falls back to any available language.
2	timedtext XML API	Directly queries <code>video.google.com/timedtext</code> ; discovers available language tracks from the list endpoint.
3	<i>yt-dlp</i>	Downloads subtitle metadata without video download; parses JSON3 format; falls back to VTT/SRT text parsing.
4	Whisper ASR	Downloads audio via <i>yt-dlp</i> , trims to 600 seconds, and transcribes using <i>faster-whisper</i> (tiny model, CPU, int8 quantisation).

If the detected language is not English, the transcript is split into 3000-character chunks and each chunk is translated using the Groq Llama-3.1-8b-instant model via synchronous API requests, with a 2-second rate-limit guard between chunks.

3.4.5 Preprocessing and Normalization

Before any expensive AI processing begins, the transcript is classified using Groq Llama-3 to determine whether the video is educational. Non-educational content (entertainment, music, vlogs, gaming) is rejected immediately with an informative error response. This *fails open* — if the classifier itself fails for any reason, processing continues rather than blocking legitimate requests.

3.4.6 Model Training and Inference

1. **Block formation:** Transcript segments are merged into paragraph-like blocks using pause gap detection (≥ 2 seconds between segments) and a minimum word threshold (15 words/block).

2. **Embedding:** All blocks are embedded using the *sentence-transformers* library with the `all-MiniLM-L6-v2` model (384-dimensional dense vectors).
3. **Boundary detection:** For each pair of adjacent blocks, cosine similarity is computed. A similarity below 0.5 indicates a topic shift and marks a chapter boundary.
4. **Title generation:** Each chapter title is derived from the first 8 words of the opening block of that chapter.
5. **Course generation:** The full transcript text and video title are sent to Groq Llama-3.3-70b with a structured prompt requesting a JSON course object containing lessons, summaries, key concepts, and multiple-choice quizzes with correct answers and explanations.
6. **Async pipeline:** For long videos, `POST /generate-course-async` accepts the URL, creates a job record in MongoDB with status `processing`, and returns a `job_id` immediately. The pipeline runs in a FastAPI `BackgroundTask`. The client polls `GET /course-status/{job_id}` until status becomes `completed` or `failed`.
7. **Exam and Certificate:** The exam engine fetches all quiz questions from the cached course, grades submitted answers, computes a percentage score, and stores the result. If the score meets the 70% threshold, the certificate service builds a certificate document with a UUID identifier and generates a downloadable PDF using *reportlab*.

3.4.7 Algorithm

Algorithm 1: LearnTube — Phase 1: Course Generation Pipeline

Input: YouTube URL u

Output: Structured course JSON with lessons, chapters, and quizzes

```
1  $video\_id \leftarrow extract\_video\_id(u)$ 
2  $cached \leftarrow MongoDB.get\_course(video\_id)$ 
3 if  $cached \neq \emptyset$  then return  $cached$  // cache hit -- skip pipeline
4
   // Transcript waterfall
5  $result \leftarrow \emptyset$ 
6 for  $strategy$  in [ $youtube\_transcript\_api$ ,  $timedtext$ ,  $yt-dlp$ ,  $Whisper$ ] do
7    $result \leftarrow strategy(video\_id)$ 
8   if  $result.segments \neq \emptyset$  then break
9 end
10 if  $result.language \neq English$  then
    $result.transcript \leftarrow Groq\_Translate(result.transcript, chunk = 3000)$ 
   // Content classification
11  $category \leftarrow Groq\_Classify(result.title, result.transcript)$ 
12 if  $category \neq \text{"educational"}$  then return error: non-educational content
   // Semantic chapter detection
13  $blocks \leftarrow merge\_segments(result.segments, gap = 2s, min\_words = 15)$ 
14  $embeddings \leftarrow SentenceTransformer.encode(blocks)$ 
15  $chapters \leftarrow []$ 
16 for  $i \leftarrow 1$  to  $len(blocks) - 1$  do
17    $sim \leftarrow cosine\_similarity(embeddings[i - 1], embeddings[i])$ 
18   if  $sim < 0.5$  then
      $chapters.append(\{title : blocks[i][: 8 \text{ words}], time : blocks[i].start\})$ 
19 end
   // AI course generation
20  $course \leftarrow Groq\_Llama3(transcript = result.transcript, title = result.title)$ 
21  $MongoDB.cache\_course(video\_id, course, chapters)$ 
22 return  $\{course, chapters, video\_id, title\}$ 
```

Algorithm 2: LearnTube — Phase 2: Exam Grading and Certificate Issuance

Input: $video_id, user_id, user_name$, answers list A **Output:** Exam result; PDF certificate if passed

```
1  $course \leftarrow \text{MongoDB.get\_course}(video\_id)$ 
2  $quiz\_map \leftarrow \{quiz.id \mapsto quiz : quiz \in course.lessons.quizzes\}$ 
3  $correct \leftarrow 0$ 
4 for  $submission$  in  $A$  do
5    $quiz \leftarrow quiz\_map[submission.quiz\_id]$ 
6   if  $submission.selected == quiz.correctAnswer$  then  $correct \leftarrow correct + 1$ 
7 end
8  $score \leftarrow \frac{correct}{|A|} \times 100$ 
9  $passed \leftarrow (score \geq 70)$ 
10  $\text{MongoDB.save\_exam}(user\_id, video\_id, score, passed)$ 
11 if  $\neg passed$  then return  $\{score, passed=False, results\}$ 
    // Certificate generation
12  $cert \leftarrow \text{build\_certificate}(user\_id, user\_name, video\_id, course.title)$ 
13  $pdf \leftarrow \text{reportlab.generate}(cert)$ 
14  $\text{MongoDB.save\_certificate}(cert)$ 
15 return  $\{certificate\_id, pdf\_url, course\_title, completion\_date\}$ 
```

CHAPTER 4

RESULTS AND DISCUSSION

4.1 Training and Validation Progress

LearnTube was evaluated through comprehensive functional testing across a wide range of video types and scenarios. The system was validated on English and non-English videos, caption-less videos requiring ASR fallback, non-educational content that should be rejected, cached video re-requests, long-form async processing, and the complete exam-to-certificate lifecycle. All 12 test scenarios passed successfully.

The course generation pipeline demonstrated a steady and reliable performance across all tested domains including Python programming, DBMS, calculus, English grammar, physics lectures, and competitive exam preparation. The transcript extraction waterfall correctly escalated through strategies only when needed, and the Groq-based content classifier correctly identified and rejected non-educational content such as music videos.

4.2 Performance Evaluation Metrics

Table-3 : Functional test results

Test Scenario	Expected Outcome	Result
English video with auto-captions	Course generated with lessons and quizzes	PASS
English video with manual captions	Course generated with lessons and quizzes	PASS
Hindi video (non-English transcript)	Translated to English; course generated	PASS
Spanish video	Translated to English; course generated	PASS
Caption-less video (Whisper ASR fallback)	ASR transcript; course generated	PASS
Non-educational video (music)	Rejected with informative error	PASS
Same video requested twice	Served from cache ($< 1s$)	PASS
Long video (> 1 hour) via async end-point	Job created; result polled successfully	PASS
Exam submission with score $\geq 70\%$	Certificate PDF issued	PASS

Test Scenario	Expected Outcome	Result
Exam submission with score < 70%	Failure response; no certificate issued	PASS
Certificate retrieval by ID	PDF returned via shareable link	PASS
Invalid YouTube URL	Error response; no pipeline triggered	PASS

4.3 Confusion Matrix Analysis

The overall quantitative performance metrics are summarised below:

Metric	Result
Transcript extraction success rate	~95% for videos with available captions
Average course generation time (sync)	20–45 seconds
Cache hit response time	< 1 second
Chapter detection (10+ videos)	Visually accurate boundaries
Exam grading latency	< 1 second (fully automated)
Certificate PDF generation	< 2 seconds
Async pipeline (long videos)	Zero timeout failures
Multilingual pipeline (Hindi/Spanish/French)	Successfully translated and structured

4.4 Comparison with Existing Approaches

Table-4 : Feature comparison with existing systems

Feature	<i>LearnTube</i>	<i>Coursera/Udemy</i>	<i>YouTube</i>	<i>Summify</i>	<i>ChatGPT (manual)</i>
End-to-end from YouTube URL	✓	×	×	✓	×
Structured lessons & modules	✓	✓	×	×	Partial
Auto quiz generation	✓	✓	×	×	Manual
Certificate issuance	✓	✓	×	×	×
Multilingual auto-translation	✓	×	×	×	Manual
Semantic chapter detection	✓	×	×	×	×
Async processing (long videos)	✓	N/A	N/A	×	×
MongoDB caching	✓	N/A	N/A	×	×
Free to use	✓	×	✓	Partial	Partial
Zero manual input required	✓	N/A	N/A	✓	×

CHAPTER 5

CONCLUSION AND FUTURE WORK

5.1 Conclusion

This project demonstrates that converting a raw educational YouTube video into a complete, certifiable learning experience is practically achievable in a single automated pipeline. The following conclusions can be drawn from the design, implementation, and evaluation of LearnTube:

1. **Robust transcript extraction is achievable without a single point of failure.** The four-strategy waterfall — *youtube-transcript-api*, *timedtext* API, *yt-dlp*, and Whisper ASR — achieves approximately 95% success across diverse video types. The ASR fallback using *faster-whisper* with CPU int8 quantisation ensures that even caption-less videos can be processed without specialised hardware.
2. **Semantic chapter detection using sentence embeddings is practical and effective.** The `all-MiniLM-L6-v2` cosine similarity approach correctly identifies major topic shifts in educational transcripts without any task-specific fine-tuning. The 0.5 similarity threshold generalises well across different academic domains.
3. **Groq-powered LLM course generation produces pedagogically useful output.** The Llama-3.3-70b model reliably generates structured course JSON with coherent lessons, accurate summaries, and plausible quiz questions with explanations. The output quality is sufficient for self-directed learning and exam preparation.
4. **The asynchronous job architecture fully resolves the long-video timeout problem.** The BackgroundTasks-based async pipeline with MongoDB job status tracking eliminates all timeout failures for videos exceeding 30 minutes, which would otherwise be unprocessable in a synchronous HTTP pipeline.
5. **The complete learning lifecycle is achievable with zero manual effort from the learner.** From URL entry through course study, graded exam, and PDF certificate issuance, every step is fully automated. The learner provides only a YouTube URL and their exam answers; all other processing is handled by the system.

5.2 Limitations

While the project established a solid foundation for automated course generation, several constraints were identified during evaluation:

- **Caption Dependency:** Approximately 5% of videos fail transcript extraction even after all four strategies are attempted, typically due to completely inaccessible or corrupted captions combined with very poor audio quality for ASR.
- **LLM Output Variability:** The Groq Llama-3.3-70b model occasionally generates quiz questions that are too simplistic or that repeat concepts across lessons, particularly for very short or poorly structured videos.
- **Translation Quality:** Chunk-based translation of non-English transcripts may lose contextual coherence at chunk boundaries for highly technical content.
- **Vocabulary Constraints:** The content classifier may incorrectly classify highly specialized or niche technical videos as non-educational.
- **Operational Requirements:** The system requires an active Groq API key and MongoDB instance; no fully offline mode is currently available.

5.3 Future Work

To transition this research into a more practical, widely deployed application, the following improvements are proposed:

1. **Multi-Video Playlist Support.** Extend the pipeline to accept a YouTube playlist URL and aggregate all videos in the playlist into a single structured curriculum with ordered modules and cross-video progress tracking.
2. **Adaptive Quiz Difficulty.** Implement a learner performance model that adjusts the difficulty of quiz questions based on historical answer patterns, providing an increasingly personalised assessment experience.
3. **PDF Downloadable Study Notes.** Generate print-ready, formatted PDF study notes for every lesson in a course, including key concepts, summaries, and definitions, downloadable for offline study.
4. **Learning Analytics Dashboard.** Build a per-user analytics interface tracking time-on-lesson, quiz accuracy trends, chapter completion heatmaps, and overall learning velocity across all generated courses.

5. **Sentence-Level Translation Integration.** Integrating Large Language Models to compose sequences of recognized words into grammatically coherent outputs, improving multilingual course coherence.
6. **Blockchain-Based Certificate Verification.** Issue certificates as on-chain commitments (e.g., Ethereum ERC-721 or a simple registry contract) to provide tamper-proof, publicly verifiable credentials that learners can share with employers or institutions.
7. **Mobile Application.** Develop a native iOS/Android application (React Native) with offline course access, push notifications for learning reminders, and platform-native authentication.
8. **LMS Platform Integration.** Export generated courses in IMS Common Cartridge or xAPI (Tin Can) format for direct import into Moodle, Canvas, or other institutional LMS platforms, enabling LearnTube to serve as a content generation layer for formal education systems.

REFERENCES

- [1] Reimers, N. & Gurevych, I. (2019). *Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks*. Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP). <https://arxiv.org/abs/1908.10084>
- [2] Radford, A., Kim, J.W., Xu, T., Brockman, G., McLeavey, C., & Sutskever, I. (2022). *Robust Speech Recognition via Large-Scale Weak Supervision (Whisper)*. OpenAI Technical Report. <https://arxiv.org/abs/2212.04356>
- [3] de Poix, J. (2023). *youtube-transcript-api: A Python API for retrieving YouTube transcripts*. GitHub Repository. <https://github.com/jdepoix/youtube-transcript-api>
- [4] yt-dlp contributors (2023). *yt-dlp: A feature-rich command-line audio/video downloader*. GitHub Repository. <https://github.com/yt-dlp/yt-dlp>
- [5] Groq Inc. (2024). *Groq API Documentation: Llama-3 Inference*. <https://console.groq.com/docs>
- [6] Ramírez, S. (2023). *FastAPI: High Performance, Easy to Learn, Fast to Code, Ready for Production*. <https://fastapi.tiangolo.com>
- [7] Vercel Inc. (2024). *Next.js Documentation: The React Framework for Production*. <https://nextjs.org/docs>
- [8] UKP Lab, TU Darmstadt (2023). *sentence-transformers: Multilingual Sentence, Paragraph, and Image Embeddings using BERT*. <https://www.sbert.net>
- [9] MongoDB Inc. (2024). *MongoDB Documentation: Atlas and Motor Async Driver*. <https://www.mongodb.com/docs>
- [10] ReportLab Inc. (2023). *ReportLab: PDF Generation Library for Python*. <https://www.reportlab.com/docs>
- [11] Devlin, J., Chang, M., Lee, K., & Toutanova, K. (2019). *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*. Proceedings of NAACL-HLT 2019. <https://arxiv.org/abs/1810.04805>
- [12] SYSTRAN (2023). *faster-whisper: Faster Whisper transcription with CTranslate2*. GitHub Repository. <https://github.com/SYSTRAN/faster-whisper>